

AWS/GCP 실전 마스터 과정 - 3일차

1교시: 클라우드 비용 구조와 서비스 과금 체계 이해

개념 설명

AWS와 GCP 모두 **종량제(pay-as-you-go)** 요금 체계를 채택하여 사용한 만큼만 비용을 지불합니다. 예를 들어 AWS는 수도·전기 요금처럼 사용한 서비스에 대해서만 과금하며, 서비스를 중단하면 추가 요금이나 해지 비용이 발생하지 않습니다. 또한 장기 사용을 약정할 경우 요금을 할인해 주는데, AWS는 1년 또는 3년 약정의 **Reserved Instances** 또는 **Savings Plans**를 제공하여 컴퓨팅 비용을 절감해 줍니다. AWS S3나 데이터 전송 비용처럼 사용량이 많아질수록 GB당 요금이 낮아지는 **계층형(tiered)** 요금도 적용합니다.

GCP도 마찬가지로 초기 비용이나 해지 수수료 없이 사용량 기준으로만 비용을 청구합니다. GCP는 **커밋 약정(Committed Use)**을 통해 장기 약정 시 최대 57%까지 할인해 주며, 예산·알림 설정, AI 기반 권장사항 등 다양한 비용 관리 도구를 제공합니다. 예를 들어 구글 클라우드 콘솔에서는 예산을 설정해 비용 경고를 받을 수 있고, 맞춤형 권장사항을 통해 사용량을 최적화하도록 안내합니다.

또한 두 클라우드 모두 신규 사용자에게 무료 혜택을 제공합니다. AWS는 신규 계정에 최대 **\$200 크레딧**과 **6개월간 일부 서비스 무료** 체험을 제공하며, EC2·S3 등을 포함한 30여 개 서비스에 월별 사용 한도 내에서 **Always Free(항상 무료)** 혜택을 제공합니다 ¹. GCP는 가입 시 **\$300 크레딧(90일간 유효)**을 지급하고, 20여 개의 서비스(예: 1개의 f2-micro 인스턴스, 5GB 스토리지, BigQuery 쿼리 등)를 월별 한도 내에서 영구 무료로 이용할 수 있습니다. [아키텍처 다이어그램 삽입]

실습 목표 및 배경 시나리오

- **실습 목표:** AWS와 GCP의 과금 정책을 이해하고, 클라우드 비용 계산기(Calculator)와 비용 관리 도구를 이용해 비용을 예측합니다. 비용을 최적화하기 위한 할인 옵션과 예산 관리 방법을 실습합니다.
- **배경 시나리오:** 예비 창업자 A씨는 클라우드로 웹 서비스를 개발하려고 합니다. 초기 비용을 최소화하기 위해 AWS와 GCP의 무료 혜택을 활용하며, 예상 사용량에 따른 비용을 예측하여 예산을 수립하고자 합니다.

실습 절차

1. **계정 및 비용 계산기 준비:** AWS 콘솔에 로그인하고 AWS Pricing Calculator에 접속합니다. GCP 콘솔에 로그인하고 Google Cloud 가격 계산기에 접속합니다.
2. **요금 구조 검토:** AWS Pricing 문서와 GCP Pricing 개요를 참고하여 각 서비스의 요금 구조를 살펴봅니다. CPU, 메모리, 스토리지, 네트워크 등 주요 서비스의 단가를 확인합니다.
3. **비용 예측 계산:** 웹 서비스 구성(예: EC2 2대, RDS 1대, S3 100GB, 데이터 전송 등)을 가정하고, 두 클라우드 계산기에 각각 입력하여 월별 예상 요금을 계산합니다. 할인 요금제(Reserved/Committed Use) 옵션을 적용해 보며 비용 절감 효과를 비교합니다.
4. **비용 관리 도구 사용:** AWS Cost Explorer 및 GCP Billing 콘솔에서 예산(Budgets)과 경고(Alerts)를 설정해 봅니다. 예를 들어 월별 예산 한도를 설정하고 초과 시 이메일 알림을 받도록 합니다.

실습 코드 예시 (AWS & GCP 병렬 예시)

```
# AWS: CLI로 비용 예측 (Cost Explorer API)
aws ce get-cost-forecast \
```

```
--time-unit MONTHLY \
--metric BLENDED_COST \
--time-period Start=2025-01-01,End=2025-04-01

# AWS: 예산 경고 생성 (AWS Budgets API)
aws budgets create-budget \
--account-id 123456789012 \
--budget file://budget.json
```

```
# GCP: CLI로 예산 생성 (Cloud Billing Budgets)
gcloud alpha billing budgets create \
--billing-account BLD-123456-789ABC \
--display-name="TestBudget" \
--budget-amount=1000 \
--threshold-rule=percent=0.8

# GCP: 명령줄로 가격 계산 (예: VM 생성시 예상 요금 조회)
gcloud compute instances create dummy-instance \
--zone=asia-northeast1-a \
--machine-type=e2-medium
```

실습 결과 예측

- AWS와 GCP 계산기를 이용해 입력한 리소스의 예상 월별 비용이 출력됩니다. (예: [스크린샷 예시 삽입])
- 할인 요금제(Reserved/Committed Use) 적용 시 예상 비용이 감소하는 것을 확인할 수 있습니다.
- 설정한 예산(Budgets) 기준으로 비용이 예측치를 초과하면 콘솔에서 경고 알림(이메일)이 생성됩니다.

요약 박스

- **혼자 해보기:** 실제 서비스를 구상하여 AWS/GCP 가격 계산기를 이용해 월별 요금을 스스로 계산해 보세요. Discounts 옵션(저장 계획·커밋 약정)을 적용해 보고, 사용량이 많은 서비스에 대한 계층 할인(tiered pricing)을 확인하세요.
- **퀴즈:** AWS의 Savings Plans와 GCP의 Committed Use Discount 차이는 무엇인가요? 클라우드 프리 티어(Free Tier)의 조건은 각각 어떻게 되나요?
- **체크리스트:** 클라우드 서비스 과금 방식(pay-as-you-go, 예약 요금제, 프리 티어)을 이해했나요? 비용 계산기 및 비용 관리 도구(예산, 경고)를 실습해 보았나요? 할인 옵션과 자동 스케일링이 비용에 미치는 영향을 확인했나요?

2교시: 클라우드 과금 예측 및 리소스 비용 최적화 실습

개념 설명

클라우드 사용량과 비용을 정확히 예측하고 관리하는 것은 운영 비용 절감과 서비스 안정성 확보에 필수적입니다. AWS는 **Cost Explorer**와 **Budgets**를 통해 과거 사용량 데이터를 분석하고 미래 지출을 예측할 수 있도록 지원합니다. Cost Explorer는 지난 12개월의 비용과 사용 데이터를 기반으로 향후 12개월 예측치를 제공합니다. 이렇게 생성한 예측값을 토대로 예산 한도를 설정하고 초과 여부를 모니터링할 수 있습니다. AWS Budgets를 사용하면 일/월/연 단위로 예산을 설정하고, 실제 비용 혹은 예측 비용이 한도에 근접하거나 초과할 때 이메일 알림을 받을 수 있습니다.

GCP는 **Cloud Billing Reports**와 **Budgets & alerts** 기능으로 비용 추세를 시각화하고 제어합니다. 콘솔 내 예산 도구에서 프로젝트별 예산을 설정하고 초과 시 알림을 받을 수 있으며, **비용 권장사항(Cost Recommendations)**을 통해 불필요한 리소스를 식별할 수 있습니다. 또한 GCP의 인공지능 기반 권장사항(Intelligent Recommendations)은 권장되는 최적화 조치를 안내하여 비용 절감을 돕습니다.

실습 목표 및 배경 시나리오

- **실습 목표:** AWS Cost Explorer와 GCP 비용 관리 툴을 사용하여 비용 예측과 모니터링을 실시하고, 불필요한 리소스를 찾아 비용을 최적화하는 방법을 배웁니다.
- **배경 시나리오:** 스타트업 B사는 서비스 사용량 증가로 비용 부담이 커지는 것을 우려합니다. 과거 데이터로 향후 비용을 예측하고, 권장사항 도구를 통해 과도한 자원 사용을 식별하여 비용을 절감하려고 합니다.

실습 절차

1. **비용 탐색기 활성화 (AWS):** AWS 콘솔에서 Cost Explorer를 활성화합니다. 활성화 후 24시간 이내에 과거 비용 데이터가 준비됩니다.
2. **비용 예측:** Cost Explorer에서 EC2, S3 등 주요 서비스의 월별 비용 그래프를 확인하고, 향후 3개월 예측 그래프를 생성합니다. 예측 데이터를 기반으로 월별 예산을 설정합니다.
3. **예산 및 알림 설정:** AWS Budgets에서 월별 비용 예산을 생성하고, 예산의 80% 및 100% 초과 시 이메일 알림을 설정합니다.
4. **비용 최적화 권장사항 확인:** AWS Trusted Advisor 또는 AWS Compute Optimizer를 사용하여 미사용 리소스(예: 아이들링 EC2 인스턴스, 과도한 용량의 RDS 등)와 비용 절감 권장사항을 확인합니다. 필요한 경우 인스턴스 타입 변경이나 예약 인스턴스 전환을 고려합니다.
5. **GCP 예산 및 권장사항:** GCP 콘솔에서 Billing → Budgets & alerts로 이동하여 비슷한 예산을 생성하고, Cloud Recommender(권장사항) 페이지에서 절감 가능한 리소스를 검토합니다. 예를 들어, 사용량이 적은 머신타입이나 비활성화된 VM 인스턴스가 감지될 수 있습니다.

실습 코드 예시 (AWS & GCP 병렬 예시)

AWS: Cost Explorer를 통한 예측치 조회 (API 사용 예시)

```
aws ce get-forecast \
  --time-unit MONTHLY \
  --metric BLENDED_COST \
  --time-period Start=2024-10-01,End=2025-01-01
```

AWS: 예산 조회 (AWS Budgets)

```
aws budgets describe-budgets \
  --account-id 123456789012
```

AWS: 미사용 EC2 확인 (CloudWatch 지표 사용)

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-0abc123def456ghij \
  --start-time 2025-01-01T00:00:00Z --end-time 2025-01-07T00:00:00Z \
  --statistics Average
```

GCP: 예산 생성 (gcloud CLI)

```
gcloud alpha billing budgets create \
```

```
--billing-account=B0B0-B0B0-B0B0 \
--display-name="MonthlyBudget" \
--amount=500 \
--threshold-rule=percent=0.8

# GCP: 권장사항 조회 (비용 절감 권장사항)
gcloud recommender recommendations list \
--project=my-project-id \
--recommender=google.compute.instance.MachineTypeRecommender

# GCP: VM 사용량 확인 (Cloud Monitoring API 예시)
gcloud monitoring metrics list --filter="metric.type=compute.googleapis.com/instance/
cpu/utilization"
```

실습 결과 예측

- Cost Explorer에서 선택한 기간의 실제 비용 그래프와 함께 다음 달 예측 비용이 그래프로 표시됩니다.
- 예산 경고 설정에 따라 예산 80% 초과 시 이메일 알림이 전송됩니다. (예: [\[스크린샷 예시 삽입\]](#))
- Trusted Advisor 또는 GCP Recommender의 권장사항 목록에 절감 가능한 리소스가 표시됩니다. 인스턴스 타입 변경, 예약 사용 전환 등의 권장작업이 제시됩니다.

요약 박스

- **혼자 해보기:** 과거 한 달간의 비용 데이터를 바탕으로 AWS Cost Explorer에서 분기별 예측을 생성해 보세요. GCP에서는 Cloud Billing Reports로 동일한 분석을 수행해 보세요. 또 다른 권장사항 도구(예: Azure Recommender)와 비교해 보는 것도 좋습니다.
- **퀴즈:** AWS Trusted Advisor의 **Cost Optimization** 카테고리에는 포함되는 주요 권장사항은 무엇인가요? GCP Cloud Recommender에서 제공하는 권장사항 종류는 어떤 것들이 있나요?
- **체크리스트:** 예산(Budgets)을 설정하여 초과 알림을 받도록 했나요? Cost Explorer나 Billing Reports로 비용 추세를 시각화했나요? 권장사항에서 확인된 비효율적인 자원을 반영하여 인스턴스 규격을 변경하거나 예약 사용을 적용했나요?

3교시: CloudWatch / Cloud Monitoring을 활용한 서비스 모니터링 실습

개념 설명

서비스 안정성과 성능 유지를 위해 **모니터링**은 필수입니다. AWS의 **CloudWatch**는 EC2, RDS, Lambda 등 AWS 리소스의 기본 지표(CPU, 메모리, 디스크 I/O 등)를 수집하여 대시보드에 시각화하고, 임계값 초과 시 알람을 발생시켜줍니다 ² . 필요에 따라 애플리케이션에서 커스텀 메트릭을 게시할 수도 있습니다. CloudWatch는 수집한 모든 메트릭을 15개월간 보관하므로, 과거 데이터와 실시간 데이터를 모두 분석할 수 있습니다 ² .

GCP의 **Cloud Monitoring**(이전 명칭 Stackdriver)은 GCP 및 AWS 리소스의 성능, 가용성, 지연시간 등의 지표를 종합적으로 모니터링하는 통합 서비스입니다. Cloud Monitoring은 VM 인스턴스, Kubernetes 클러스터, 빅데이터 처리 등 다양한 소스의 지표를 수집하여 사용자 정의 대시보드로 시각화합니다. 또한 알람 정책(Alerting Policy)을 설정해 특정 지표가 임계값을 초과하면 이메일이나 SMS 등의 방식으로 알람을 보낼 수 있습니다. 이처럼 CloudWatch와 Cloud Monitoring 모두 기본 모니터링 지표 수집, 그래프 생성, 알람 기능을 제공하며, 서비스 가용성과 성능을 체계적으로 관리할 수 있게 해줍니다 ² .

실습 목표 및 배경 시나리오

- **실습 목표:** AWS CloudWatch와 GCP Cloud Monitoring을 사용하여 가상 머신의 리소스 사용률을 모니터링하고, 임계치를 설정해 자동 알람을 구성합니다.
- **배경 시나리오:** 스타트업 C사는 웹 애플리케이션의 CPU 사용률이 일정 수준을 초과할 때 자동으로 알람을 받고 싶어 합니다. 한편, 운영자는 애플리케이션의 로그에서 여러 메시지를 기반으로 알람을 받도록 설정하여 장애를 조기에 감지하려고 합니다.

실습 절차

1. **리소스 생성:** AWS에서는 EC2 인스턴스(예: t3.micro) 하나를 생성하고, GCP에서는 Compute Engine VM 인스턴스(예: f1-micro)를 생성하여 각각 실행 중인 상태로 둡니다.
2. **기본 지표 확인:** AWS 콘솔의 CloudWatch 대시보드에서 EC2 인스턴스의 CPUUtilization, NetworkIn/Out 지표를 확인합니다. GCP 콘솔의 Monitoring 대시보드(또는 Metrics Explorer)에서 VM 인스턴스의 cpu/utilization, disk/write_time 등을 확인합니다.
3. **임계치 기반 알람 설정 (AWS):** AWS CLI 또는 콘솔을 사용해 CloudWatch Alarm을 생성합니다. 예를 들어 CPUUtilization이 80%를 연속 5분 동안 초과하면 알람이 발생하도록 설정합니다.
4. **임계치 기반 알람 설정 (GCP):** GCP에서는 Cloud Monitoring에서 알람 정책을 생성합니다. VM의 CPU 사용률(metric: `compute.googleapis.com/instance/cpu/utilization`)이 80% 이상일 때 알람이 발생하도록 구성합니다. Notification 채널로 이메일이나 Pub/Sub를 연결합니다.
5. **로그 기반 알람 설정:** (선택사항) AWS CloudWatch Logs로 EC2 시스템 로그를 전송하고, 에러 문자열이 감지되면 알람이 발생하도록 Log Alarm을 만듭니다. GCP Stackdriver Logging에서도 유사하게 로그 기반 알람을 설정할 수 있습니다.

실습 코드 예시 (AWS & GCP 병렬 예시)

```
# AWS: CloudWatch Alarm 생성 예시 (CLI)
aws cloudwatch put-metric-alarm \
  --alarm-name HighCPUAlarm \
  --metric-name CPUUtilization --namespace AWS/EC2 \
  --statistic Average --period 300 --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 \
  --dimensions Name=InstanceId,Value=i-0abcd1234efgh5678 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:MyTopic

# AWS: 로그 기반 알람 생성 (CLI 예시, 로그 그룹과 필터 설정 필요)
aws logs put-metric-filter \
  --log-group-name /aws/ec2/instance/logs \
  --filter-name ErrorMetric \
  --filter-pattern "ERROR" \
  --metric-transformations
metricName=ErrorCount,metricNamespace=Custom,metricValue=1
aws cloudwatch put-metric-alarm \
  --alarm-name ErrorAlarm \
  --metric-name ErrorCount --namespace Custom \
  --statistic Sum --period 300 --threshold 1 \
  --comparison-operator GreaterThanThreshold \
```

```
--evaluation-periods 1 \
--alarm-actions arn:aws:sns:us-east-1:123456789012:MyTopic
```

GCP: Cloud Monitoring 알람 정책 생성 예시 (gcloud CLI 사용)

```
gcloud monitoring policies create \
--notification-channels=projects/my-project/notificationChannels/123 \
--display-name="High CPU Alert" \
--conditions='displayName=High CPU,
conditionThreshold={filter="metric.type=\"compute.googleapis.com/instance/cpu/utilization\"",
comparison=COMPARISON_GT, thresholdValue=0.8, duration="300s"}'
```

GCP: 간단한 VM 생성 및 지표 조회

```
gcloud compute instances create my-vm \
--zone=asia-northeast1-a --machine-type=e2-micro
```

Cloud Monitoring에서는 기본으로 VM의 cpu/utilization 메트릭이 자동 수집됩니다.

실습 결과 예측

- 설정한 CPU 임계치(예: 80%)를 초과하면 AWS CloudWatch와 GCP Cloud Monitoring 모두 알람이 트리거됩니다. 콘솔의 알람 페이지나 이메일/SMS를 통해 알람을 확인할 수 있습니다. (예: [스크린샷 예시 삽입])
- AWS 로그 기반 알람의 경우, EC2 로그에 “ERROR” 문자열이 기록되면 ErrorAlarm이 발생합니다.
- 두 플랫폼 모두 모니터링 차트와 알람 상태를 확인하여 정상, 경고 상태를 판별할 수 있습니다. 예를 들어 CloudWatch에서는 Alarm 탭에 ‘ALARM’ 상태가 표시됩니다.

요약 박스

- **혼자 해보기:** CloudWatch 대시보드에서 직접 그래프를 생성해 보세요. GCP에서는 Metrics Explorer를 이용해 커스텀 지표를 그래프로 그려보고, 알람 정책을 실습해 보세요.
- **퀴즈:** AWS CloudWatch와 GCP Cloud Monitoring의 가장 큰 차이점은 무엇인가요? 각 서비스에서 기본 제공되는 모니터링 지표와 커스텀 지표 수집 방식을 비교해 보세요.
- **체크리스트:** 모니터링할 리소스(EC2/GCE)를 생성했나요? CloudWatch/Monitoring에서 지표를 확인했나요? 임계치 알람을 생성하고 테스트했나요? 로그 기반 알람도 설정해 보셨나요?

4교시: 종합 실습 - 컨테이너 자동 배포 + 로드밸런싱 + 오토스케일링 + 비용 모니터링 + 복구 전략 설계

개념 설명

현대적인 클라우드 운영에서는 **컨테이너 기반** 애플리케이션의 자동 배포와 오토스케일링, 로드밸런싱이 핵심입니다. AWS의 **EKS(Elastic Kubernetes Service)**와 GCP의 **GKE(Google Kubernetes Engine)**는 매니지드 쿠버네티스 서비스로, 컨테이너 클러스터를 손쉽게 운영할 수 있게 해줍니다. 이들은 노드 풀을 기반으로 자동으로 컴퓨팅 자원을 확장/축소하고, 서비스 앞단에 **로드 밸런서**를 통합하여 트래픽 분산을 지원합니다 ³ ⁴. 예를 들어 AWS EKS는 **카펜터(Karpenter)**를 이용해 필요할 때 자동으로 EC2 노드를 추가/제거하고, AWS Elastic Load Balancing(ALB/NLB)과 연동하여 서비스의 가용성과 비용 효율을 높입니다 ³ ⁵. GKE 역시 클러스터 오토스케일러가 사용량에 따라 노드를 자동 조절하고, Kubernetes Service 타입에 따라 Google Cloud Load Balancer를 자동 생성합니다. **LoadBalancer** 타입 서비스는 TCP/UDP L4 로드밸런서를, **Ingress** 리소스는 HTTP(S) L7 로드밸런서를 각각 프로비저닝합니다 ⁴ ⁶.

운영 측면에서 여러 AZ(가용 영역)에 걸쳐 배포하면 장애 복구 능력이 향상됩니다. 또한, 비용 모니터링을 위해 CloudWatch나 Cloud Monitoring 지표를 사용하여 실시간 사용량을 점검하고, 예산을 초과하지 않도록 관리해야 합니다. 종합 실습에서는 컨테이너 애플리케이션을 클러스터에 자동 배포한 후 로드밸런싱과 오토스케일링을 구성하고, 앞서 배운 비용 관리 기법과 복구 전략(예: 멀티 AZ, 데이터 백업)을 포함한 아키텍처를 설계합니다. [아키텍처 다이어그램 삽입]

실습 목표 및 배경 시나리오

- **실습 목표:** EKS/GKE를 사용하여 컨테이너 기반 서비스를 자동 배포하고, 로드 밸런서와 오토스케일링을 구성합니다. 또한 CloudWatch/Monitoring을 통해 성능과 비용을 모니터링하고, 장애 발생 시 복구 전략을 설계합니다.
- **배경 시나리오:** 스타트업 D사는 컨테이너로 구현한 웹 서비스를 AWS와 GCP 환경에 배포하려 합니다. 사용량 급증 시 자동 확장을 원하며, 장애가 발생해도 자동 복구 되도록 하고 싶습니다. 또한 예산 내에서 운영 비용을 모니터링해야 합니다.

실습 절차

1. **컨테이너 이미지 준비:** 간단한 웹 애플리케이션(예: nginx, Node.js 앱 등)의 Docker 이미지를 준비하고 AWS ECR과 GCP Container Registry(Artifact Registry)에 푸시합니다.
2. **클러스터 생성:** AWS에서 `eksctl` 또는 AWS Management Console을 통해 EKS 클러스터를 생성합니다 (`region: ap-northeast-2`, `노드수: 2`). GCP에서는 `gcloud container clusters create` 명령으로 GKE 클러스터를 생성합니다 (`zone: asia-northeast1-a`, `노드수: 2`).
3. **디플로이먼트 및 서비스 배포:** 준비한 Deployment/Service YAML을 사용해 쿠버네티스에 애플리케이션을 배포합니다. 서비스는 LoadBalancer 타입으로 설정하여 자동으로 외부 로드밸런서를 생성하도록 합니다.
4. **오토스케일링 구성:** Kubernetes HPA(Horizontal Pod Autoscaler)를 설정해 CPU 사용률(예: 50%) 기준으로 파드를 자동 확장하게 구성합니다. 클러스터 레벨에서는 EKS의 카펜터나 GKE 클러스터 오토스케일러를 활성화하여 노드 수를 자동으로 조정합니다 5 7.
5. **로드밸런싱 확인:** AWS EKS에서는 ELB가, GKE에서는 Google Cloud Load Balancer가 생성되어 서비스 트래픽을 분산합니다. 생성된 로드밸런서 DNS를 통해 서비스 접속을 확인합니다.
6. **비용 모니터링:** CloudWatch(지표, 대시보드)와 Cloud Monitoring(비용대시보드, 예산알림)을 이용해 리소스 사용량과 비용을 모니터링합니다. 예산 초과 여부를 주기적으로 확인하여 경고 알림을 설정합니다.
7. **복구 전략 설계:** 여러 AZ에 노드를 분산하여 배포하고, 애플리케이션 데이터베이스는 스냅샷/백업을 설정합니다. 장애 발생 시 자동 재배포(Auto Healing)를 위해 EKS/GKE의 관리형 노드 그룹을 사용하거나, 컨테이너 레디니스를 구성합니다.

실습 코드 예시 (AWS & GCP 병렬 예시)

AWS: EKS 클러스터 생성 (eksctl 사용 예시)

```
eksctl create cluster --name my-eks-cluster --region ap-northeast-2 \
--nodes 2 --node-type t3.small
```

Kubernetes 애플리케이션 배포 (AWS)

```
kubectl apply -f aws-deployment.yaml
```

Deployment 예시(YAML snippet):

```
# apiVersion: apps/v1
```

```
# kind: Deployment
```

```
# metadata: {name: web-app}
```

```
# spec:
```

```
# replicas: 2
```

```
# selector: {matchLabels: {app: web}}
```

```
# template:
```

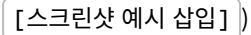


```
# metadata: {labels: {app: web}}
# spec:
#   containers:
#     - name: web
#       image: 123456789012.dkr.ecr.ap-northeast-2.amazonaws.com/my-web-app:latest
#       ports: [{containerPort: 80}]
```

```
# GCP: GKE 클러스터 생성
gcloud container clusters create my-gke-cluster \
  --zone=asia-northeast1-a --num-nodes=2 --machine-type=e2-small

# Kubernetes 애플리케이션 배포 (GCP)
kubectl apply -f gcp-deployment.yaml
# Deployment 예시(YAML snippet):
# apiVersion: apps/v1
# kind: Deployment
# metadata: {name: web-app}
# spec:
#   replicas: 2
#   selector: {matchLabels: {app: web}}
#   template:
#     metadata: {labels: {app: web}}
#     spec:
#       containers:
#         - name: web
#           image: asia.gcr.io/my-project/my-web-app:latest
#           ports: [{containerPort: 80}]
```

실습 결과 예측

- EKS/GKE 클러스터가 생성되고, Kubernetes Service가 외부 로드밸런서를 프로비저닝합니다. AWS의 경우 ELB DNS를, GCP의 경우 gcloud에서 부여된 IP/DNS를 통해 접속이 가능합니다. (예: )
- 트래픽 부하에 따라 HPA가 파드를 자동으로 늘리거나 줄입니다. 이를 통해 CPU 사용률이 설정 임계치를 넘으면 자동으로 파드가 추가되고, 부하 감소 시 최소 복제수로 줄어듭니다. 클러스터 오토스케일러는 노드 수를 조정하여 자원 활용을 최적화합니다 ⁷.
- CloudWatch/Monitoring 대시보드에 노드와 파드의 CPU/메모리 사용량이 표시되며, 예산 설정 시 비용 초과 시점에 알림이 발생합니다. 멀티 AZ 구성 덕분에 하나의 AZ가 장애가 나도 다른 AZ에서 서비스가 지속됩니다.

요약 박스

- **혼자 해보기:** 다른 예제로 Kubernetes 애플리케이션(예: 백엔드+프론트엔드 2개 서비스)을 구성해 보세요. Pod Disruption Budget, Readiness Probe를 설정하여 안정성을 테스트해 보고, 노드 장애 시 복구가 제대로 이루어지는지 실험해 보세요.
- **퀴즈:** AWS EKS와 GCP GKE의 네트워크 로드밸런싱 방식을 비교해 보세요. NLB와 ALB를 언제 사용하는지, GKE의 Ingress와 Service LoadBalancer 차이점은 무엇인지 설명해 보세요.
- **체크리스트:** 컨테이너 이미지와 레지스트리를 구성했나요? 클러스터 생성 후 애플리케이션이 정상 기동되었나요? 오토스케일링(HPA, 클러스터)과 로드밸런싱 설정을 확인했나요? 예산/모니터링 알림을 설정했나요? 멀티 AZ와 백업 전략을 고려했나요?

1 Free Cloud Computing Services - AWS Free Tier

<https://aws.amazon.com/free/>

2 Metrics in Amazon CloudWatch - Amazon CloudWatch

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/working_with_metrics.html

3 5 Automate cluster infrastructure with EKS Auto Mode - Amazon EKS

<https://docs.aws.amazon.com/eks/latest/userguide/automode.html>

4 6 About load balancing in GKE | GKE networking | Google Cloud

<https://cloud.google.com/kubernetes-engine/docs/concepts/about-load-balancing>

7 About GKE cluster autoscaling | Google Kubernetes Engine (GKE) | Google Cloud

<https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-autoscaler>